JobAI Scout Voice Assistant

Architecture, implementation, data flow, rules, testing, and operations

Purpose

The JobAI Scout Voice Assistant lets an authenticated user speak a career question, receive a Gemini-powered answer, and hear that answer spoken aloud. It can also use the user's uploaded documents as a private retrieval-augmented generation (RAG) knowledge base.

High-Level Architecture

```
Browser microphone -> browser speech recognition -> Gemini transcription fallback ->
Gemini embedding and RAG retrieval -> Gemini chat answer -> Gemini text-to-speech ->
browser playback and history storage.
```

Layer	Component	Responsibility
Client	VoiceMode	Records microphone input, detects end of speech, requests transcription/chat/TTS, and plays audio.
Client	VoiceRecognition	Uses Chrome or Edge Web Speech API first for fast local transcript capture.
Edge	voice-transcribe	Uses Gemini only when browser speech recognition has no final transcript.
Edge	voice-chat	Authenticates the user, retrieves relevant knowledge chunks, calls Gemini, persists messages, and cache
Edge	voice-tts	Converts the final answer to speech using Gemini TTS.
Database	kb_sources / kb_chunks	Stores uploaded source metadata, chunk text, and vector embeddings for RAG.
Storage	voice-history	Stores optional user and assistant audio after the response path is already running.

How a Voice Turn Works

1. The user taps the microphone. The browser requests microphone permission and starts both MediaRecorder and browser speech recognition when available.

2. The voice activity detector waits for actual speech. It does not end a turn immediately because the user paused before talking.

3. After speech begins, 2.2 seconds of silence ends the turn. If the user never speaks, the microphone ends after 3 seconds with a clear no-speech message. A second tap while listening stops recording immediately.

4. The browser transcript is used first. If there is no final browser transcript, the recording is sent to Gemini transcription.

5. The text question is sent to voice-chat. Short conversations avoid an unnecessary query-rewrite request to reduce latency.

6. voice-chat produces a Gemini embedding, searches the user's private knowledge-base chunks, adds relevant context, and requests a concise Gemini answer.

7. The final answer is sent to Gemini TTS and played immediately. Background history uploads do not block the response.

RAG Document Flow

A PDF, TXT, DOC, DOCX, Markdown, or CSV file is uploaded through the Voice Assistant knowledge button.

kb-ingest-document extracts readable text, cleans it, splits it into bounded chunks, and creates a Gemini embedding for every chunk.

The source record is stored in kb_sources and the text plus vectors are stored in kb_chunks. A ready source means the document is available for retrieval.

For a voice question, the same embedding model creates a query vector. Database functions retrieve the closest chunks for that authenticated user only.

Implementation Decisions

Gemini is the AI provider for transcription fallback, embeddings, chat, and TTS. The former OpenRouter audio-balance dependency was removed from the user-facing voice path.

Browser speech recognition is preferred because it avoids sending audio to the server and usually returns text faster than server transcription.

Storage writes are asynchronous after the response begins. This protects perceived response speed while retaining history when storage is available.

Voice answers are capped at 480 output tokens. Voice replies should be concise, useful, and natural to hear.

Knowledge retrieval is scoped to the logged-in user through database policies and user identifiers passed to the matching functions.

Rules and Safety Controls

Authentication: voice-chat requires an authenticated Supabase user before it reads history, documents, or writes messages.

Data isolation: RAG retrieval only searches the current user's knowledge-base chunks. The assistant must not expose another user's documents, profile, or history.

Secrets: Gemini and service credentials are stored as Supabase secrets or ignored local environment values. They must never be placed in client code, browser storage, logs, or this document.

Consent: microphone access depends on browser permission. The user controls recording by tapping the microphone again.

No fabrication: if context is missing, the assistant should not invent document content, pricing, platform policies, or private information.

Input defense: prompts are sanitized before the chat request, and the assistant has career-focused guardrails.

Fallback behavior: no browser transcript uses Gemini transcription; a Gemini TTS failure falls back to browser speech synthesis; no speech shows a clear retry message.

Testing Performed

Gemini text generation was verified with a minimal server-side request.

Gemini embedding generation was verified at the configured 1,536 vector dimensions.

Document RAG ingestion was verified through a successful uploaded-document ready state and the kb_sources / kb_chunks storage flow.

The kb-ingest-document bug was corrected: insert results are now read directly instead of calling select() on an already resolved result object.

voice-transcribe, voice-chat, voice-agent-llm, and kb-ingest-document were deployed after their relevant changes.

The frontend production build completed successfully after voice flow changes.

Voice-flow regression checks confirm the 3-second no-speech timeout, second-click stop control, removed obsolete OpenRouter error text, and latency settings.

Manual Acceptance Test

1. Sign in, open Dashboard -> Voice Assistant, and allow microphone access.

2. Tap the microphone, speak one short question, then remain silent. The turn should end after about 2.2 seconds of post-speech silence.

3. Confirm that the assistant starts processing and plays an answer. Repeat with a question whose answer exists only in an uploaded document.

4. In Supabase, verify kb_sources shows status ready and kb_chunks contains rows with embeddings for the uploaded file.

5. Tap the microphone and do not speak. It should stop after 3 seconds and show a no-speech message. Tap while listening to stop immediately.

Operational Notes

The first Gemini request can take longer than later requests because of provider cold-start and network latency. Browser transcript, concise answers, caching, and non-blocking history writes reduce the normal path delay.

A live microphone test still requires a real signed-in browser session with microphone permission. Automated validation can verify builds, functions, embeddings, and API behavior, but cannot speak into the user's microphone.